

MANUAL OPERATIVO SGA SaaS

Comandos entregados, finalidad y operación segura

Producción: <https://sgaholding.online>

VPS: Ubuntu 24.04 - Docker - Caddy - PostgreSQL 16 - Cloudflare R2

Release documentada: v1.0.5 / commit b2e7204

Clasificación: documento operativo privado. Incluye rutas, dominio e IP del VPS. No contiene contraseñas, tokens, secretos JWT, claves Fernet ni credenciales R2 reales.

Actualizado: domingo 9 de agosto de 2026 - Zona horaria: America/Bogota

Índice

Índice	2
1. Cómo utilizar este manual	5
2. Resumen del entorno	5
3. Repositorio, GitHub y publicación	5
3.1 Entrar al repositorio	5
3.2 Commit y push	5
3.3 Crear release	5
3.4 Secretos Docker Hub	6
3.5 Validaciones de CI y release	6
4. Acceso y diagnóstico del VPS	6
4.1 Conectarse por SSH	6
4.2 Inventario del sistema	6
4.3 Puertos y firewall	7
4.4 DNS	7
4.5 Clave SSH de despliegue	7
5. Estructura y transferencia	7
5.1 Crear directorios	7
5.2 Copiar archivos	7
5.3 Permisos	7
5.4 Actualización con rollback	8
6. Secretos y archivo .env	8
6.1 Generar secretos	8
6.2 Crear .env	8
6.3 Variables esenciales	9
6.4 Mostrar datos no secretos	9
6.5 Validar Compose	9

7. Despliegue de producción	9
7.1 Desplegar	9
7.2 Inicializar roles	9
7.3 Verificar sga_app	9
7.4 Estado de contenedores	10
8. Verificaciones HTTPS y seguridad	10
8.1 Readiness HTTPS	10
8.2 Rutas públicas y API	10
8.3 Redirección www	10
8.4 Logs de errores	10
8.5 Cabeceras de seguridad	10
9. Cloudflare R2	11
9.1 Capturar credenciales	11
9.2 Variables R2 en .env	11
9.3 Recrear solo backend	11
9.4 Validar configuración R2	11
9.5 Prueba R2 CRUD	12
10. Backups de aplicación	12
10.1 Versión pg_dump	12
10.2 Backup manual completo	12
10.3 Incidentes corregidos	12
10.4 Verificar sga_backup	12
10.5 Verificar read-only	13
11. Automatización diaria	13
11.1 Activar backups	13
11.2 Aplicar scheduler	13
11.3 Calcular primera ejecución	13
11.4 Consultar backup automático	13

12. Limpieza local y retención	14
12.1 Probar limpiador	14
12.2 Programar cron	14
12.3 Verificar cron y log	14
13. Copia cifrada de configuración	14
13.1 Abrir script	14
13.2 Ejecutar	14
13.3 Confirmar archivo	14
13.4 Descargar a Windows	15
13.5 Verificar SHA-256	15
14. Operación diaria	15
14.1 Estado general	15
14.2 Recursos	15
14.3 Logs	15
14.4 Reiniciar solo backend	15
14.5 Cerrar SSH	16
15. Errores frecuentes	16
16. Lista final de seguridad	16
Apéndice A. Script cifrado de un solo uso	16
Apéndice B. Referencia rápida	17

1. Cómo utilizar este manual

Este documento consolida los comandos utilizados durante la preparación del repositorio, publicación de imágenes, configuración del VPS, despliegue, seguridad, Cloudflare R2, backups y verificaciones. Los bloques repetidos se muestran una sola vez.

- PowerShell: comandos ejecutados en la computadora Windows.
- VPS/Bash: comandos ejecutados después de conectarse como sgaadmin.
- CAMBIAR_..., TU_... y <...> son marcadores; nunca deben copiarse literalmente a producción.
- Nunca compartir .env, contraseñas, tokens, claves privadas SSH ni secretos R2

Regla crítica: la barra invertida continúa líneas en Bash/Linux. En PowerShell se usa el acento grave. Los comandos Linux no se ejecutan directamente en PowerShell.

2. Resumen del entorno

Componente	Valor operativo
Dominio	sgaholding.online / www.sgaholding.online
VPS	2.25.97.17 - sgaadmin - SSH 22
Sistema	Ubuntu 24.04.4 LTS, x86_64
Contenedores	sga_postgres, sga_backend, sga_frontend y caddy
Release	v1.0.5 - b2e72048630c4decf80d026c2dd3ae496d40efc2
R2	Bucket privado sga-production-backups; prefijo sga-production/
Retención	30 días local; 45 días en R2

3. Repositorio, GitHub y publicación

3.1 Entrar al repositorio

Para qué sirve: Evita el error de ejecutar Git desde una carpeta que no es repositorio.

Dónde se ejecuta: PowerShell

```
cd C:\Proyectos\SGA_SaaS
git status --short
git rev-parse --short HEAD
```

Resultado esperado: Git responde y muestra el commit corto.

Precaución: Si aparece 'not a git repository', ejecutar primero `cd C:\Proyectos\SGA_SaaS`.

3.2 Commit y push

Para qué sirve: Añade cambios revisados, crea un commit descriptivo y publica main.

Dónde se ejecuta: PowerShell

```
git add -- &lt;archivo1> &lt;archivo2>
git commit -m "fix(...): descripción"
git push origin main
git status
```

Resultado esperado: Working tree clean.

Precaución: Revisar que .env, credenciales, backups y evidencias privadas no estén incluidos.

3.3 Crear release

Para qué sirve: Crea una versión inmutable que dispara la construcción de imágenes.

Dónde se ejecuta: PowerShell, solo con CI verde

```
git tag -a v1.0.5 -m "v1.0.5: backup seguro con rol dedicado y RLS"
```

```
git push origin v1.0.5
```

Resultado esperado: GitHub publica el tag y ejecuta Release immutable images.

Precaución: No mover ni reutilizar tags ya publicados.

3.4 Secretos Docker Hub

Para qué sirve: Autoriza a GitHub Actions para publicar las imágenes.

Dónde se ejecuta: GitHub > Settings > Secrets and variables > Actions

```
DOCKERHUB_USERNAME=vanstrahlen
```

```
DOCKERHUB_TOKEN=&lt;TOKEN_GENERADO_EN_DOCKER_HUB&gt;
```

Resultado esperado: docker/login-action inicia sesión correctamente.

Precaución: El token no es el usuario ni la contraseña de Docker Hub; debe generarse como access token.

3.5 Validaciones de CI y release

Backend:

```
python -m compileall -q app alembic tests
```

```
python -m unittest discover -s tests -v
```

Frontend:

```
npm ci
```

```
npm run audit:prod
```

```
npm run verify:no-rsc
```

```
npm test
```

```
npm run lint
```

```
npm run build
```

Release:

```
docker/build-push-action@v6
```

```
aquasecurity/trivy-action@v0.36.0
```

```
SBOM y provenance
```

- d04c48f: Trivy Action se corrigió a v0.36.0.
- 27b6ab3: NGINX_IMAGE se declaró antes del primer FROM.
- b059d78: dependencias e imágenes vulnerables se actualizaron.
- 5a068d5: postgresql-client-16 alineó pg_dump con PostgreSQL 16.
- b2e7204: se añadió el rol sga_backup para RLS.

4. Acceso y diagnóstico del VPS

4.1 Conectarse por SSH

Para qué sirve: Abre una sesión segura en el servidor.

Dónde se ejecuta: PowerShell

```
ssh -p 22 sgaadmin@2.25.97.17
```

Resultado esperado: Prompt sgaadmin@srv1882404:~\$.

Precaución: Nunca compartir la clave privada SSH.

4.2 Inventario del sistema

Para qué sirve: Comprueba identidad, sistema, recursos, sudo y Docker.

Dónde se ejecuta: VPS/Bash

```
echo "=== USUARIO ==="
```

```
whoami
```

```
hostname
```

```
grep -E '^(PRETTY_NAME|VERSION_ID)= ' /etc/os-release
```

```
uname -m
```

```
free -h
```

```
df -h /
```

```
sudo -n true 2&gt;/dev/null && echo "SUDO_SIN_PASSWORD=SI" || echo "SUDO_SIN_PASSWORD=NO"
```

```
docker --version
```

```
docker compose version
```

Resultado esperado: Ubuntu 24.04, x86_64 y Docker/Compose disponibles.

4.3 Puertos y firewall

Para qué sirve: Revisa listeners y reglas de entrada.

Dónde se ejecuta: VPS/Bash

```
ss -lnt
sudo ufw status verbose
```

Resultado esperado: UFW deny incoming y ALLOW para 22, 80 y 443.

4.4 DNS

Para qué sirve: Confirma que dominio principal y www apuntan al VPS.

Dónde se ejecuta: VPS/Bash

```
resolvectl query sgaholding.online
resolvectl query www.sgaholding.online
```

Resultado esperado: Ambos nombres resuelven a 2.25.97.17.

4.5 Clave SSH de despliegue

Para qué sirve: Genera un par de claves dedicado.

Dónde se ejecuta: VPS/Bash

```
install -d -m 700 ~/.ssh
ssh-keygen -t ed25519 -C "sga-saas-vps-2026-08-07" -f ~/.ssh/sga_saas_deploy -N ""
chmod 600 ~/.ssh/sga_saas_deploy
chmod 644 ~/.ssh/sga_saas_deploy.pub
cat ~/.ssh/sga_saas_deploy.pub
```

Resultado esperado: Se crean la clave privada y la pública .pub.

Precaución: En formularios se pega la línea completa de .pub; nunca la clave privada.

5. Estructura y transferencia

5.1 Crear directorios

Para qué sirve: Prepara la ruta productiva y asigna propiedad.

Dónde se ejecuta: VPS/Bash

```
sudo mkdir -p /opt/sga_saas/backend/docker
sudo chown -R sgaadmin:sgaadmin /opt/sga_saas
cd /opt/sga_saas
```

Resultado esperado: /opt/sga_saas pertenece a sgaadmin.

Precaución: mkdir -p es sintaxis Linux, no PowerShell.

5.2 Copiar archivos

Para qué sirve: Transfiere los archivos operativos desde Windows.

Dónde se ejecuta: PowerShell desde el repositorio

```
scp -P 22 .\docker-compose.prod.yml sgaadmin@2.25.97.17:/opt/sga_saas/
scp -P 22 .\Caddyfile sgaadmin@2.25.97.17:/opt/sga_saas/
scp -P 22 .\deploy.sh sgaadmin@2.25.97.17:/opt/sga_saas/
scp -P 22 .\env.example sgaadmin@2.25.97.17:/opt/sga_saas/
scp -P 22 .\backend\docker\init-app-role.sh sgaadmin@2.25.97.17:/opt/sga_saas/backend/docker/
```

Resultado esperado: Cada archivo termina en 100%.

5.3 Permisos

Para qué sirve: Protege scripts y configuración.

Dónde se ejecuta: VPS/Bash

```
chmod 750 deploy.sh backend/docker/init-app-role.sh
chmod 640 docker-compose.prod.yml Caddyfile .env.example
ls -la
```

Resultado esperado: Scripts ejecutables y configuración no pública.

5.4 Actualización con rollback

Para qué sirve: Sustituye archivos conservando la versión anterior.

Dónde se ejecuta: VPS/Bash

```
cp -p docker-compose.prod.yml docker-compose.prod.yml.pre-v1.0.5
cp -p backend/docker/init-app-role.sh backend/docker/init-app-role.sh.pre-v1.0.5
mv docker-compose.prod.yml.v1.0.5 docker-compose.prod.yml
mv backend/docker/init-app-role.sh.v1.0.5 backend/docker/init-app-role.sh
chmod 640 docker-compose.prod.yml
chmod 750 backend/docker/init-app-role.sh
sh -n backend/docker/init-app-role.sh && echo "SCRIPT_ACTIVADO_VALIDO"
docker compose --env-file .env -f docker-compose.prod.yml config --quiet && echo "COMPOSE_ACTIVADO_VALIDO"
```

Resultado esperado: Ambas validaciones terminan en VALIDO.

6. Secretos y archivo .env

6.1 Generar secretos

Para qué sirve: Crea contraseñas diferentes y claves criptográficas en memoria.

Dónde se ejecuta: VPS/Bash

```
umask 077
OWNER_PASS="$(python3 -c 'import secrets; print(secrets.token_urlsafe(36))')"
APP_PASS="$(python3 -c 'import secrets; print(secrets.token_urlsafe(36))')"
BACKUP_PASS="$(python3 -c 'import secrets; print(secrets.token_urlsafe(36))')"
JWT_SECRET="$(python3 -c 'import secrets; print(secrets.token_urlsafe(48))')"
FERNET_KEY="$(python3 -c 'import base64,secrets;
    print(base64.urlsafe_b64encode(secrets.token_bytes(32)).decode())')"
printf 'OWNER_PASS=%s caracteres\n' "${#OWNER_PASS}"
printf 'APP_PASS=%s caracteres\n' "${#APP_PASS}"
printf 'BACKUP_PASS=%s caracteres\n' "${#BACKUP_PASS}"
printf 'JWT_SECRET=%s caracteres\n' "${#JWT_SECRET}"
printf 'FERNET_KEY=%s caracteres\n' "${#FERNET_KEY}"
```

Resultado esperado: Longitudes aproximadas 48, 48, 48, 64 y 44.

Precaución: No imprimir valores reales ni enviarlos por chat.

6.2 Crear .env

Para qué sirve: Copia la plantilla, limita permisos y abre el editor.

Dónde se ejecuta: VPS/Bash

```
cp .env.example .env
chmod 600 .env
nano .env
```

Resultado esperado: .env con permisos 600.

Precaución: Nunca ejecutar git add .env.

6.3 Variables esenciales

```
APP_ENV=production
DEBUG=false
IMAGE_TAG=&lt;SHA_COMPLETO_PUBLICADO&gt;
POSTGRES_DB=sga_db
POSTGRES_USER=sga_owner
POSTGRES_PASSWORD=&lt;SECRETO_OWNER&gt;
POSTGRES_APP_PASSWORD=&lt;SECRETO_APP&gt;
POSTGRES_BACKUP_PASSWORD=&lt;SECRETO_BACKUP&gt;
DATABASE_URL=postgresql://sga_app:&lt;SECRETO_APP&gt;@postgres:5432/sga_db
BACKUP_DATABASE_URL=postgresql://sga_backup:&lt;SECRETO_BACKUP&gt;@postgres:5432/sga_db
MIGRATION_DATABASE_URL=postgresql://sga_owner:&lt;SECRETO_OWNER&gt;@postgres:5432/sga_db
SECRET_KEY=&lt;SECRETO_JWT&gt;
CONFIG_ENCRYPTION_KEY=&lt;CLAVE_FERNET&gt;
FRONTEND_URL=https://sgaholding.online
BACKEND_CORS_ORIGINS=https://sgaholding.online
REFRESH_COOKIE_SECURE=true
RUN_MIGRATIONS=false
RUN_SCHEDULER=true
ALLOW_DATABASE_RESTORE=false
```

6.4 Mostrar datos no secretos

Para qué sirve: Comprueba modo, dominio, tag y banderas.

Dónde se ejecuta: VPS/Bash

```
grep -E '^(APP_ENV|DEBUG|IMAGE_TAG|POSTGRES_DB|POSTGRES_USER|FRONTEND_URL|BACKEND_CORS_ORIGINS|S3_BACKUP_ENABLED|REFRESH_COOKIE_SECURE|RUN_MIGRATIONS|ALLOW_DATABASE_RESTORE)=' .env
```

Resultado esperado: APP_ENV=production y DEBUG=false.

6.5 Validar Compose

Para qué sirve: Resuelve variables y sintaxis antes del despliegue.

Dónde se ejecuta: VPS/Bash

```
docker compose --env-file .env -f docker-compose.prod.yml config --quiet \
&& echo "COMPOSE_VALIDO"
```

Resultado esperado: COMPOSE_VALIDO.

7. Despliegue de producción

7.1 Desplegar

Para qué sirve: Inicia Caddy, descarga imágenes, migra y espera health checks.

Dónde se ejecuta: VPS/Bash

```
sudo ./deploy.sh
```

Resultado esperado: Servicios healthy y readiness HTTPS 200.

7.2 Inicializar roles

Para qué sirve: Crea roles cuando el volumen PostgreSQL ya existía.

Dónde se ejecuta: VPS/Bash

```
chmod 755 backend/docker/init-app-role.sh
docker exec sga_postgres /docker-entrypoint-initdb.d/10-init-app-role.sh
```

Resultado esperado: CREATE/ALTER ROLE, GRANT y ALTER DEFAULT PRIVILEGES.

7.3 Verificar sga app

Para qué sirve: Confirma que el usuario web no tiene privilegios administrativos ni BYPASSRLS.

Dónde se ejecuta: VPS/Bash

```
docker exec sga_postgres psql -U sga_owner -d sga_db -Atc \
"SELECT rolname, rolcanlogin, rolsuper, rolcreatedb, rolcreaterole, rolbypassrls FROM pg_roles WHERE
rolname='sga_app';"
```

Resultado esperado: sga_app|t|f|f|f|f.

7.4 Estado de contenedores

Para qué sirve: Comprueba servicios y migración one-shot.

Dónde se ejecuta: VPS/Bash

```
docker compose --env-file .env -f docker-compose.prod.yml ps -a
docker ps --filter "name=~/caddy$" --format "table {{.Names}} {{.Image}} {{.Status}} {{.Ports}}"
```

Resultado esperado: PostgreSQL, backend y frontend healthy; migrate Exited (0); Caddy activo.

8. Verificaciones HTTPS y seguridad

8.1 Readiness HTTPS

Para qué sirve: Confirma Caddy, frontend, backend y PostgreSQL.

Dónde se ejecuta: VPS/Bash

```
curl -fsS -o /dev/null \
  -w 'HTTPS_READY=%{http_code}' \
  \
  https://sgaholding.online/health/ready
```

Resultado esperado: HTTPS_READY=200.

8.2 Rutas públicas y API

Para qué sirve: Valida frontend y health checks.

Dónde se ejecuta: VPS/Bash

```
for url in \
  "https://sgaholding.online/" \
  "https://sgaholding.online/health/live" \
  "https://sgaholding.online/health/ready" \
  "https://sgaholding.online/api/health/live" \
  "https://sgaholding.online/api/health/ready"
do
  code="$(curl -sS -o /dev/null -w '%{http_code}' "$url")"
  printf '%s -&gt; %s\n' "$url" "$code"
done
```

Resultado esperado: Todas devuelven 200.

8.3 Redirección www

Para qué sirve: Concentra el sitio en el dominio canónico.

Dónde se ejecuta: VPS/Bash

```
curl -sS -o /dev/null \
  -w 'www -&gt; %s | redirect: %s\n' \
  \
  https://www.sgaholding.online/
```

Resultado esperado: www -> 301 | redirect: https://sgaholding.online/.

8.4 Logs de errores

Para qué sirve: Busca excepciones y problemas TLS.

Dónde se ejecuta: VPS/Bash

```
docker compose --env-file .env -f docker-compose.prod.yml logs --since 5m --no-color backend 2&gt;&1 | \
  grep -Ei 'traceback|exception|critical|error' || echo "BACKEND_SIN_ERRORES"
docker logs --since 5m caddy 2&gt;&1 | \
  grep -Ei '"level":"error"|failed|certificate error' || echo "CADDY_SIN_ERRORES"
```

Resultado esperado: BACKEND_SIN_ERRORES y CADDY_SIN_ERRORES.

8.5 Cabeceras de seguridad

Para qué sirve: Verifica HSTS, CSP y defensas del navegador.

Dónde se ejecuta: VPS/Bash

```
curl -sS -D - -o /dev/null https://sgaholding.online/ | \
  grep -Ei '^(\ HTTP/|strict-transport-security:|content-security-policy:|x-content-type-options:|x-frame-op
```

tions:|referrer-policy:|permissions-policy:)'

Resultado esperado: HTTP/2 200 y cabeceras presentes.

Precaución: Cabeceras duplicadas indican que Caddy y frontend añaden la misma política; no implica caída.

9. Cloudflare R2

El bucket es privado, se llama sga-production-backups y usa el prefijo sga-production/. El token está limitado al bucket y a la IP 2.25.97.17/32.

9.1 Capturar credenciales

Para qué sirve: Carga credenciales R2 sin mostrarlas.

Dónde se ejecuta: VPS/Bash

```
umask 077
read -rsp "Pega R2 Access Key ID y pulsa Enter: " R2_ACCESS_KEY_ID; echo
read -rsp "Pega R2 Secret Access Key y pulsa Enter: " R2_SECRET_ACCESS_KEY; echo
read -rsp "Pega el endpoint S3 completo y pulsa Enter: " R2_ENDPOINT; echo
R2_ENDPOINT="${R2_ENDPOINT%/*}"
printf 'ACCESS_KEY=%s caracteres\n' "${R2_ACCESS_KEY_ID}"
printf 'SECRET_KEY=%s caracteres\n' "${R2_SECRET_ACCESS_KEY}"
if [[ "$R2_ENDPOINT" =~ ^https://[A-Za-z0-9]+\.\r2\.cloudflarestorage\.com$ ]]; then
    echo "ENDPOINT_R2=VALIDO"
else
    echo "ENDPOINT_R2=REVISAR"
fi
```

Resultado esperado: Secret Key de 64 caracteres y endpoint válido.

Precaución: Account ID, Access Key ID y Secret Access Key son valores distintos.

9.2 Variables R2 en .env

Para qué sirve: Habilita la copia fuera del VPS.

Dónde se ejecuta: VPS/Bash, editando .env

```
S3_BACKUP_ENABLED=true
S3_BACKUP_ENDPOINT_URL=https://&lt;ACCOUNT_ID&gt;.r2.cloudflarestorage.com
S3_BACKUP_REGION=auto
S3_BACKUP_BUCKET=sga-production-backups
S3_BACKUP_ACCESS_KEY_ID=&lt;ACCESS_KEY_ID&gt;
S3_BACKUP_SECRET_ACCESS_KEY=&lt;SECRET_ACCESS_KEY&gt;
S3_BACKUP_PREFIX=sga-production
```

Resultado esperado: Backend recibe la configuración; .env sigue en modo 600.

9.3 Recrear solo backend

Para qué sirve: Aplica .env sin reiniciar los demás servicios.

Dónde se ejecuta: VPS/Bash

```
docker compose --env-file .env -f docker-compose.prod.yml up -d \
--no-deps --force-recreate --wait --wait-timeout 180 backend
```

Resultado esperado: sga_backend Healthy.

9.4 Validar configuración R2

Para qué sirve: Confirma variables dentro del contenedor.

Dónde se ejecuta: VPS/Bash

```
docker compose --env-file .env -f docker-compose.prod.yml exec -T backend \
python -c 'from app.config import settings; valid = settings.S3_BACKUP_ENABLED and
settings.S3_BACKUP_BUCKET == "sga-production-backups" and settings.S3_BACKUP_REGION == "auto" and
bool(settings.S3_BACKUP_ENDPOINT_URL) and bool(settings.S3_BACKUP_ACCESS_KEY_ID) and
bool(settings.S3_BACKUP_SECRET_ACCESS_KEY); print("BACKEND_R2_CONFIGURADO" if valid else
"BACKEND_R2_INCOMPLETO")'
```

Resultado esperado: BACKEND_R2_CONFIGURADO.

9.5 Prueba R2 CRUD

Para qué sirve: Valida escritura, lectura y eliminación de un objeto temporal.

Dónde se ejecuta: VPS/Bash

```
docker compose --env-file .env -f docker-compose.prod.yml exec -T backend python - &&'PY'
import uuid, boto3
from app.config import settings
payload = b'SGA R2 verification'
key = f'{settings.S3_BACKUP_PREFIX.strip('/')}/checks/{uuid.uuid4()}.txt'
client = boto3.client('s3', endpoint_url=settings.S3_BACKUP_ENDPOINT_URL,
    region_name=settings.S3_BACKUP_REGION,
    aws_access_key_id=settings.S3_BACKUP_ACCESS_KEY_ID,
    aws_secret_access_key=settings.S3_BACKUP_SECRET_ACCESS_KEY)
client.put_object(Bucket=settings.S3_BACKUP_BUCKET, Key=key, Body=payload)
assert client.get_object(Bucket=settings.S3_BACKUP_BUCKET, Key=key)['Body'].read() == payload
client.delete_object(Bucket=settings.S3_BACKUP_BUCKET, Key=key)
print('R2_ESCRITURA_LECTURA_ELIMINACION=OK')
PY
```

Resultado esperado: R2_ESCRITURA_LECTURA_ELIMINACION=OK.

10. Backups de aplicación

10.1 Versión pg_dump

Para qué sirve: Evita incompatibilidad cliente-servidor.

Dónde se ejecuta: VPS/Bash

```
docker compose --env-file .env -f docker-compose.prod.yml exec -T backend pg_dump --version
```

Resultado esperado: pg_dump PostgreSQL 16.14 o cliente 16 compatible.

10.2 Backup manual completo

Para qué sirve: Genera ZIP con base y uploads, registra y sube a R2.

Dónde se ejecuta: VPS/Bash

```
docker compose --env-file .env -f docker-compose.prod.yml exec -T backend python - &&'PY'
from app.database import SessionLocal
from app.services.smart_backup_service import SmartBackupService
db = SessionLocal()
try:
    backup = SmartBackupService(db).ejecutar_backup(
        tipo="MANUAL", incluir_db=True, incluir_uploads=True,
        incluir_codigo=False, creado_por="verificacion-produccion")
    metadata = backup.metadata_json or {}
    print(f"BACKUP_ESTADO={backup.estado}")
    print(f"BACKUP_ARCHIVO={backup.nombre_archivo}")
    print(f"BACKUP_BYTES={backup.tamano_bytes}")
    print("BACKUP_R2=" + ("OK" if metadata.get("remote_key") else "ERROR"))
finally:
    db.close()
PY
```

Resultado esperado: BACKUP_ESTADO=EXITOSO y BACKUP_R2=OK.

10.3 Incidentes corregidos

- Cliente pg_dump 15 contra servidor 16: se instaló postgresql-client-16.
- RLS bloqueaba sga_app: se creó sga_backup con BYPASSRLS, lectura global, límite 2 y read-only.

10.4 Verificar sga_backup

Para qué sirve: Confirma privilegios mínimos y lectura RLS.

Dónde se ejecuta: VPS/Bash

```
docker exec sga_postgres psql -U sga_owner -d sga_db -Atc \
"SELECT rolname, rolcanlogin, rolsuper, rolcreatedb, rolcreatorole, rolreplication, rolbypassrls,
    rolconnlimit, pg_has_role('sga_backup', 'pg_read_all_data', 'member') FROM pg_roles WHERE
    rolname='sga_backup';"
```

Resultado esperado: sga_backup|t|f|f|f|f|t|2|t.

10.5 Verificar read-only

Para qué sirve: Evita modificaciones accidentales con el rol de backup.

Dónde se ejecuta: VPS/Bash

```
docker exec -e PGPASSWORD="$BACKUP_PASS" sga_postgres \
  psql -h 127.0.0.1 -U sga_backup -d sga_db -Atc \
  "SELECT current_user, current_setting('default_transaction_read_only');"
unset BACKUP_PASS
```

Resultado esperado: sga_backup|on.

Evidencia: sga_backup_20260807_214130.zip quedó íntegro, con PostgreSQL y uploads; tamaño y SHA-256 coinciden con R2.

11. Automatización diaria

11.1 Activar backups

Para qué sirve: Habilita intervalo diario y retención local.

Dónde se ejecuta: VPS/Bash

```
docker exec sga_postgres psql -U sga_owner -d sga_db -v ON_ERROR_STOP=1 -c \
"UPDATE automatizaciones
SET activo=TRUE, frecuencia_minutos=1440, estado='ACTIVO',
mensaje='Backup diario de producción habilitado',
configuracion='{\"hora\":\"02:00\", \"retencion_dias\":30, \"incluir_db\":true, \"incluir_uploads\":true, \"incluir_codigo\":false}':json,
proxima_ejecucion=NOW()+INTERVAL '1 day', actualizado_en=NOW()
WHERE modulo='backups';"
```

Resultado esperado: UPDATE 1.

Precaución: hora=02:00 es informativa; la versión actual usa un IntervalTrigger de 24 horas.

11.2 Aplicar scheduler

Para qué sirve: Relee las automatizaciones activas.

Dónde se ejecuta: VPS/Bash

```
docker compose --env-file .env -f docker-compose.prod.yml up -d \
--no-deps --force-recreate --wait --wait-timeout 180 backend
```

Resultado esperado: Backend healthy.

Precaución: Reiniciar backend reinicia el intervalo de 24 horas.

11.3 Calcular primera ejecución

Para qué sirve: Suma 24 horas al inicio real del contenedor.

Dónde se ejecuta: VPS/Bash

```
TZ=America/Bogota date '+AHORA_COLOMBIA=%Y-%m-%d %H:%M:%S %Z'
BACKEND_STARTED="$(docker inspect -f '{{.State.StartedAt}}' sga_backend)"
BACKEND_EPOCH="$(date -d "$BACKEND_STARTED" +%s)"
TZ=America/Bogota date -d "@$BACKEND_EPOCH" '+BACKEND_INICIO_COLOMBIA=%Y-%m-%d %H:%M:%S %Z'
TZ=America/Bogota date -d "@$((BACKEND_EPOCH + 86400))" '+PRIMER_BACKUP_ESTIMADO=%Y-%m-%d %H:%M:%S %Z'
```

Resultado esperado: Estimado documentado: 2026-08-09 18:40:22 -05.

11.4 Consultar backup automático

Para qué sirve: Muestra estado, R2 y horas Colombia.

Dónde se ejecuta: VPS/Bash, después de 2026-08-09 18:50

```
docker exec sga_postgres psql -U sga_owner -d sga_db -P pager=off -c "
SELECT tipo, estado, nombre_archivo, tamaño_bytes, creado_por,
CASE WHEN COALESCE(metadata-&gt;&gt;'remote_key','')&lt;&gt;' THEN 'OK' ELSE 'NO' END AS r2,
iniciado_en AT TIME ZONE 'America/Bogota' AS inicio_colombia,
finalizado_en AT TIME ZONE 'America/Bogota' AS final_colombia, mensaje
FROM backup_historial WHERE tipo='AUTOMATICO'
ORDER BY iniciado_en DESC LIMIT 5;"
```

Resultado esperado: AUTOMATICO | EXITOSO | scheduler | OK.

Precaución: Antes de la hora prevista, (0 rows) es normal.

12. Limpieza local y retención

12.1 Probar limpiador

Para qué sirve: Ejecuta la limpieza manual de copias mayores de 30 días.

Dónde se ejecuta: VPS/Bash

/opt/sga_saas/scripts/cleanup-local-backups.sh

Resultado esperado: BACKUPS_LOCALES_ELIMINADOS=0 o cantidad eliminada.

12.2 Programar cron

Para qué sirve: Ejecuta el limpiador diariamente a las 03:15 Colombia.

Dónde se ejecuta: VPS/Bash

```
(crontab -l 2>&t;/dev/null; echo '15 3 * * * /opt/sga_saas/scripts/cleanup-local-backups.sh &&t; /opt
/sga_saas/logs/backup-cleanup.log 2>&t;&1 # SGA_BACKUP_CLEANUP') | crontab -
```

Resultado esperado: Entrada instalada.

Precaución: Comprobar antes que no exista para evitar duplicados.

12.3 Verificar cron y log

Para qué sirve: Confirma demonio activo y revisa ejecución nocturna.

Dónde se ejecuta: VPS/Bash

```
crontab -l | grep 'SGA_BACKUP_CLEANUP'
systemctl is-active cron
tail -n 100 /opt/sga_saas/logs/backup-cleanup.log
```

Resultado esperado: Cron active y registro de limpieza.

Política: 30 días de retención local y 45 días en R2. La regla R2 conserva 15 días adicionales.

13. Copia cifrada de configuración

13.1 Abrir script

Para qué sirve: Crea un script temporal de un solo uso.

Dónde se ejecuta: VPS/Bash

nano /home/sgaadmin/crear-copia-sga-una-vez.sh

Resultado esperado: Nano abierto.

Precaución: La contraseña se coloca una sola vez; no usar la contraseña del VPS ni compartirla.

13.2 Ejecutar

Para qué sirve: Cifra, verifica y elimina el script con la contraseña.

Dónde se ejecuta: VPS/Bash

```
chmod 700 /home/sgaadmin/crear-copia-sga-una-vez.sh
/home/sgaadmin/crear-copia-sga-una-vez.sh
```

Resultado esperado: ARCHIVO_CIFRADO_VERIFICADO y SHA-256.

13.3 Confirmar archivo

Para qué sirve: Comprueba copia reciente, permisos y eliminación del script.

Dónde se ejecuta: VPS/Bash

```
LATEST_BACKUP="$(find /opt/sga_saas/private-backup -maxdepth 1 -type f -name 'sga-config-*.tar.gz.enc'
-printf '%T@ %p'
' | sort -nr | head -n 1 | cut -d' ' ' -f2-)"
if [[ -n "$LATEST_BACKUP" ]]; then
echo "COPIA_CIFRADA_ENCONTRADA"
```

```
printf 'BACKUP_FILE=%s
' "$LATEST_BACKUP"
stat -c 'PERMISOS=%a TAMANO=%s bytes FECHA=%y' "$LATEST_BACKUP"
sha256sum "$LATEST_BACKUP"
fi
[[ ! -e /home/sgaadmin/crear-copia-sga-una-vez.sh ]] && echo "SCRIPT_CON_CONTRASENA_ELIMINADO=OK"
```

Resultado esperado: Permisos 600 y script eliminado.

13.4 Descargar a Windows

Para qué sirve: Guarda copia fuera del VPS.

Dónde se ejecuta: PowerShell

```
New-Item -ItemType Directory -Force "C:\Users\Aorus\SGA_Backups"
scp -P 22 sgaadmin@2.25.97.17:/opt/sga_saas/private-backup/sga-config-2026-08-08_223856.tar.gz.enc
"C:\Users\Aorus\SGA_Backups\"
```

Resultado esperado: Transferencia 100%.

13.5 Verificar SHA-256

Para qué sirve: Demuestra que la copia Windows es idéntica.

Dónde se ejecuta: PowerShell

```
Get-FileHash "C:\Users\Aorus\SGA_Backups\sga-config-2026-08-08_223856.tar.gz.enc" -Algorithm SHA256
```

Resultado esperado: 04E2AACEF717D6B29DAD907FB24E0E303D07300E44079EC0B7B7E649A0037F8F.

14. Operación diaria

14.1 Estado general

Para qué sirve: Comprueba servicios sin modificarlos.

Dónde se ejecuta: VPS/Bash

```
cd /opt/sga_saas
docker compose --env-file .env -f docker-compose.prod.yml ps -a
docker ps --filter "name=^/caddy$"
```

Resultado esperado: Servicios principales healthy.

14.2 Recursos

Para qué sirve: Detecta presión de memoria o disco.

Dónde se ejecuta: VPS/Bash

```
free -h
df -h /
docker system df
```

Resultado esperado: Recursos con margen suficiente.

14.3 Logs

Para qué sirve: Obtiene errores recientes.

Dónde se ejecuta: VPS/Bash

```
docker compose --env-file .env -f docker-compose.prod.yml logs --tail=100 backend
docker compose --env-file .env -f docker-compose.prod.yml logs --tail=100 postgres
docker logs --tail=100 caddy
```

Resultado esperado: Sin traceback ni errores repetitivos.

14.4 Reiniciar solo backend

Para qué sirve: Aplica variables o recupera el servicio web.

Dónde se ejecuta: VPS/Bash

```
docker compose --env-file .env -f docker-compose.prod.yml up -d \
--no-deps --force-recreate --wait --wait-timeout 180 backend
```

Resultado esperado: Backend healthy.

Precaución: El reinicio reprograma el intervalo automático.

14.5 Cerrar SSH

Para qué sirve: Cierra la terminal sin detener el VPS.

Dónde se ejecuta: VPS/Bash

exit

Resultado esperado: Docker, cron, Caddy y scheduler continúan.

15. Errores frecuentes

Mensaje	Interpretación
not a git repository	Entrar primero a C:\Proyectos\SGA_SaaS.
sh no se reconoce	El comando es Linux; ejecutarlo dentro del VPS.
Unable to resolve Trivy	Usar aquasecurity/trivy-action@v0.36.0.
NGINX_IMAGE no declarado	Declarar ARG antes del FROM.
pg_dump version mismatch	Usar cliente PostgreSQL 16.
RLS bloquea pg_dump	Usar sga_backup, no sga_app.
R2 secret 0 caracteres	Se pulsó Enter vacío; capturar otra vez.
Endpoint R2 revisar	Usar URL S3 completa sin barra final.
AUTOMATICO 0 rows	Normal antes de la hora estimada; después revisar logs.

16. Lista final de seguridad

- CI y release verdes antes de cambiar IMAGE_TAG.
- .env en modo 600 y fuera de Git.
- APP_ENV=production y DEBUG=false.
- sga_app restringido; sga_backup read-only con BYPASSRLS.
- Migraciones en servicio migrate.
- HTTPS y health/ready 200.
- UFW solo 22, 80 y 443.
- R2 privado y token limitado.
- Retención local 30 días y R2 45 días.
- Copia cifrada fuera del VPS con SHA-256 verificado.
- Prueba de restauración futura en entorno aislado.

Apéndice A. Script cifrado de un solo uso

Sustituir solamente COLOCA_AQUI_TU_NUEVA_CONTRASENA. Usar mínimo 20 caracteres y no incluir comilla simple. El script se elimina al finalizar.

```
#!/usr/bin/env bash
set -Eeuo pipefail
umask 077
PROJECT_DIR="/opt/sga_saas"
BACKUP_DIR="${PROJECT_DIR}/private-backup"
TIMESTAMP="$(date '+%Y-%m-%d_%H%M%S') "
BACKUP_FILE="${BACKUP_DIR}/sga-config-${TIMESTAMP}.tar.gz.enc"
TEMP_BACKUP_FILE="${BACKUP_FILE}.tmp"
TEMP_DIR="$(mktemp -d)"
SCRIPT_PATH="$(readlink -f "$0")"
BACKUP_ENC_PASS='COLOCA_AQUI_TU_NUEVA_CONTRASENA'
```



```
cleanup() {
    unset BACKUP_ENC_PASS
    rm -rf -- "$TEMP_DIR"
    rm -f -- "$TEMP_BACKUP_FILE"
}
trap cleanup EXIT

[[ "$BACKUP_ENC_PASS" != "COLOCA_AQUI_TU_NUEVA_CONTRASENA" ]] || {
    echo "ERROR: debes cambiar la contraseña"; exit 1; }
(( ${#BACKUP_ENC_PASS} &gt;= 20 )) || {
    echo "ERROR: mínimo 20 caracteres"; exit 1; }

required_files=(.env docker-compose.prod.yml Caddyfile deploy.sh \
    backend/docker/init-app-role.sh scripts/cleanup-local-backups.sh)
for relative_file in "${required_files[@]"; do
    [[ -f "$PROJECT_DIR/$relative_file" ]] || {
        echo "ERROR: no existe $relative_file"; exit 1; }
done

mkdir -p "$BACKUP_DIR"
chmod 700 "$BACKUP_DIR"
mkdir -p "$TEMP_DIR/backend/docker" "$TEMP_DIR/scripts" "$TEMP_DIR/metadata"
install -m 600 "$PROJECT_DIR/.env" "$TEMP_DIR/.env"
install -m 640 "$PROJECT_DIR/docker-compose.prod.yml" "$TEMP_DIR/docker-compose.prod.yml"
install -m 640 "$PROJECT_DIR/Caddyfile" "$TEMP_DIR/Caddyfile"
install -m 750 "$PROJECT_DIR/deploy.sh" "$TEMP_DIR/deploy.sh"
install -m 750 "$PROJECT_DIR/backend/docker/init-app-role.sh" "$TEMP_DIR/backend/docker
/init-app-role.sh"
install -m 750 "$PROJECT_DIR/scripts/cleanup-local-backups.sh" "$TEMP_DIR/scripts
/cleanup-local-backups.sh"

crontab -l &gt; "$TEMP_DIR/metadata/crontab.txt" 2&gt;&1 || true
docker compose --env-file "$PROJECT_DIR/.env" -f "$PROJECT_DIR/docker-compose.prod.yml" \
    ps -a &gt; "$TEMP_DIR/metadata/docker-compose-ps.txt"
docker ps -a &gt; "$TEMP_DIR/metadata/docker-ps.txt"
(
    cd "$PROJECT_DIR"
    sha256sum .env docker-compose.prod.yml Caddyfile deploy.sh \
        backend/docker/init-app-role.sh scripts/cleanup-local-backups.sh
) &gt; "$TEMP_DIR/metadata/source-files-sha256.txt"

export BACKUP_ENC_PASS
tar -C "$TEMP_DIR" -czf - . | openssl enc -aes-256-cbc -salt -pbkdf2 \
    -iter 600000 -pass env:BACKUP_ENC_PASS -out "$TEMP_BACKUP_FILE"
openssl enc -d -aes-256-cbc -pbkdf2 -iter 600000 -pass env:BACKUP_ENC_PASS \
    -in "$TEMP_BACKUP_FILE" | tar -tzf - &gt;/dev/null
mv "$TEMP_BACKUP_FILE" "$BACKUP_FILE"
chmod 600 "$BACKUP_FILE"
echo "ARCHIVO_CIFRADO_VERIFICADO"
printf 'BACKUP_FILE=%s\n' "$BACKUP_FILE"
sha256sum "$BACKUP_FILE"
unset BACKUP_ENC_PASS
trap - EXIT
rm -rf -- "$TEMP_DIR"
command -v shred &gt;/dev/null && shred -u -- "$SCRIPT_PATH" || rm -f -- "$SCRIPT_PATH"
```

Apéndice B. Referencia rápida

Objetivo	Comando
Salud	curl -fsS -o /dev/null -w 'HTTPS_READY=%{http_code}\n' https://sgaholding.online/health/ready
Estado	docker compose --env-file .env -f docker-compose.prod.yml ps -a
Logs	docker compose --env-file .env -f docker-compose.prod.yml logs --tail=100 backend
Compose	docker compose --env-file .env -f docker-compose.prod.yml config --quiet
Cron	crontab -l grep SGA_BACKUP_CLEANUP
Recursos	free -h && df -h / && docker system df
Salir	exit

Pendiente: verificar el primer AUTOMATICO | EXITOSO | R2=OK después del domingo 9 de agosto de 2026 a las 18:50 Colombia, y revisar el log de limpieza posterior a las 03:15.